

Temporal Costmap Aggregation: Mitigating Distribution Shift in Object-Relative Visual Navigation

Kewei Liu Zihang Zhou Zihao Long

Abstract—ObjectReact [2] conditions a local navigation controller on a per-frame “WayObject Costmap” rather than raw RGB. This representation is strong when the costmap is correct, but a corrupted costmap can push the feed-forward policy away from the expert distribution with no temporal signal for recovery. We ask whether a small memory module, inserted only between the ObjectReact costmap encoder and waypoint head, can use recent costmap redundancy to make the controller more robust while preserving the original object-relative abstraction. We compare EMA, plain GRU, cosine-gated GRU, reliability-gated GRU, and reliability-aware soft fusion under a staged corruption study. A full-frame zero-out diagnostic shows that corruption-trained reliability-gated recurrence can recover from completely invalid frames, reaching 60.2 Avg SPL, but its scalar gate is not an interpretable detector: $1 - \alpha_t$ gives pooled ROC-AUC 0.312, and explicit gate supervision raises detection to ROC-AUC/AP 1.000/1.000 while collapsing navigation to 36.8/38.3 clean/noisy Avg SPL. We therefore treat detection and control as separate problems. In the main paper-aligned test, which uses per-segment outlier-cost dropout, the cosine-gated GRU is the strongest recurrent aggregator across noise rates, sweeping the 12/12 task-rate cells against the per-step baseline and reaching 42.42/21.74/19.23/26.67 SR on Imitate/Alt-Goal/Shortcut/Reverse at $\rho=0.5$. Localized patch dropout and stale-patch tests qualify the story: temporal aggregation helps, but the best aggregator depends on corruption geometry. Results use one training seed and modest episode counts, so margins are reported as trends rather than statistically established improvements.

I. INTRODUCTION

Visual navigation in indoor environments is a long-standing problem in embodied AI. Classical geometric methods rely on metric maps and specialized sensors, whereas recent learning-based methods navigate directly from images. ObjectReact [2] occupies a promising middle ground: it builds a topometric, object-level scene graph as a prior and trains a local controller on a high-level *WayObject Costmap*, a multi-channel image in which each pixel encodes the planned path length to the goal through the object it belongs to. By conditioning on object-level costs instead of an RGB subgoal image, ObjectReact decouples control from image matching and generalizes well across robot embodiments and trajectories.

The ObjectReact controller, however, is a feed-forward network that consumes *only the current* costmap. Its training data are generated by a shortest-path expert in simulation, and the policy is fit by behavior cloning. It therefore inherits the well-known failure mode of imitation learning: *distribution shift* [9]. At inference the costmap is recomputed every frame from instantaneous segmentation and matching; when the perception

pipeline mismatches segments or misses the goal, the current-frame costmap can become uninformative or misleading, and a single bad frame drives the policy toward a wrong waypoint. Critically, the ObjectReact authors themselves report this limitation: the controller uses no history context, so costmaps that vary sharply frame-to-frame destabilize the prediction, and the agent cannot recover because recovery behavior is absent from the expert demonstrations [2].

Our observation is that although any *single* costmap may be corrupted, a short window of consecutive costmaps usually carries redundant spatial cues and a smooth temporal trend—a driver who loses sight of a sign for one frame relies on the recent past rather than swerving. This motivates our central question: *can a lightweight temporal-memory module recover robustness for ObjectReact without sacrificing its cross-embodiment advantages?* Because naive averaging risks smearing a corrupted frame into the future, we also ask whether a *learned, confidence-gated* aggregator that selectively discounts anomalous frames is needed beyond ungated temporal memory.

This report formulates temporal aggregation on the WayObject Costmap and evaluates parameter-free and learned aggregators across the ObjectReact Imitate, Alt-Goal, Shortcut, and Reverse tasks. The thesis is intentionally more specific than “add recurrence”: the extension should be low-intrusion, exploit short-horizon redundancy, and leave the map, perception, expert, and waypoint head unchanged. The key scientific risk is that robustness, interpretability, and corruption realism can be conflated. A recurrent model may navigate better without learning a meaningful reliability score; a detector may become accurate without improving control; and a mechanism that handles fully invalid frames may fail under partially corrupted object segments.

We therefore keep the evaluation deliberately staged: clean runs establish the cost of memory; full-frame zero-out isolates recovery and gate semantics; explicit gate supervision tests whether calibrated detection is sufficient for control; per-segment outlier dropout matches ObjectReact’s training-time corruption; and localized patch failures test spatially coherent errors.

II. RELATED WORK

Object-relative navigation. RoboHop [1] introduced segment-based topological maps with a zero-shot controller; TANGO [7] added traversability-aware local metric control;

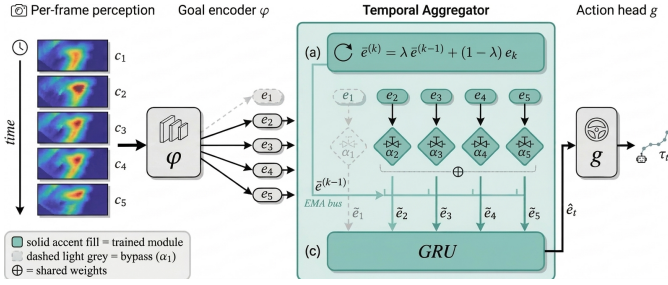


Fig. 1. Overview of our temporal costmap aggregation. We keep the ObjectReact encoder ϕ and waypoint head g warm-started from the public checkpoint and trained jointly with the aggregator, and insert an aggregator $\mathcal{A}$ over a sliding window of $K=6$ frame embeddings; the confidence-gated variants additionally down-weight frames that disagree with the intra-window history before the GRU integrates them.

ObjectReact [2] unified these with a *learned* controller conditioned on a WayObject Costmap, and is the baseline we extend. All operate on single-frame observations. The image-relative line—GNM [10] and ViNT [11]—predicts control from a current image and a retrieved subgoal image; ObjectReact adapts GNM’s pipeline but replaces the goal image with the costmap.

Imitation learning and distribution shift. Behavior cloning compounds errors once the agent leaves the expert distribution [9]. Remedies include on-policy aggregation (DAgger [9]) and history-conditioned recurrent policies [5]. Our intervention is in the latter family, applied at the costmap-feature level rather than to raw observations.

Temporal modeling in navigation. End-to-end policies commonly stack LSTM/GRU units over a visual encoder [6]; GNM [10] fuses a short RGB history in its observation encoder, a mechanism ObjectReact dropped for its goal encoder. To our knowledge, no prior work applies a temporal aggregator—let alone a confidence-gated one—to an object-relative costmap representation; this is the gap we target.

III. METHODOLOGY / SYSTEM DESIGN

A. Background: the ObjectReact controller

Let $c_t \in \mathbb{R}^{H \times W \times D}$ be the WayObject Costmap at step t ($H=64, W=85, D=8$ sine-cosine cost channels). ObjectReact applies a custom ResNet goal encoder ϕ to obtain a frame embedding $e_t = \phi(c_t) \in \mathbb{R}^{1024}$, and a GNM-style head g predicts a rollout of $T=10$ future waypoints $\tau_t = g(e_t)$. We keep the *architecture* of ϕ and g , the mapping pipeline, the perception pipeline, and the shortest-path expert generator unchanged, warm-starting ϕ and g from the public checkpoint and fine-tuning them jointly with the aggregator; our intervention is confined to the step between ϕ and g (Fig. 1).

B. Temporal costmap aggregation

We replace the single embedding e_t with an aggregate $\hat{e}_t$ computed over a sliding window of $K=6$ frames, $\{e_{t-K+1}, \dots, e_t\}$. The implementation uses `context_size=5` and follows the upstream convention

$$K = \text{context_size} + 1:$$

$$\tau_t = g(\hat{e}_t), \quad \hat{e}_t = \mathcal{A}(e_{t-K+1}, \dots, e_t). \quad (1)$$

We compare the following aggregators $\mathcal{A}$.

(1) *EMA*. A parameter-free baseline computes an exponentially weighted average over the window with decay $\lambda=0.7$, giving the most recent frame the largest weight.

(2) *Plain GRU*. A single-layer GRU (hidden size 1024) consumes the window and we take its last hidden state, $\hat{e}_t = \text{GRU}(e_{t-K+1}, \dots, e_t)$.

(3) *Cosine-confidence-gated GRU*. Within the window we maintain an intra-window running EMA of past embeddings,

$$\begin{aligned} \bar{e}_{t-K+1} &= e_{t-K+1}, \\ \bar{e}_j &= \lambda \bar{e}_{j-1} + (1 - \lambda) e_j \quad (j = t-K+2, \dots, t) \end{aligned} \quad (2)$$

with $\lambda=0.7$. For each frame $j > t-K+1$, a scalar confidence is computed from the cosine similarity between the new frame and the running mean of the frames that preceded it inside the window,

$$\alpha_j = \sigma(w \cos(e_j, \bar{e}_{j-1}) + b), \quad (w, b) \text{ learnable}, \quad (3)$$

and the raw embedding is replaced by a convex blend *before* the GRU sees it,

$$\tilde{e}_j = \alpha_j e_j + (1 - \alpha_j) \bar{e}_{j-1}. \quad (4)$$

The first frame in the window is trusted unconditionally ($\bar{e}_{t-K+1} = e_{t-K+1}, \alpha_{t-K+1}=1$). The GRU then consumes the *gated* sequence and its last hidden state is the aggregated embedding,

$$\hat{e}_t = \text{GRU}(\tilde{e}_{t-K+1}, \tilde{e}_{t-K+2}, \dots, \tilde{e}_t). \quad (5)$$

The intuition: rather than letting a corrupted frame perturb the GRU’s hidden state *after* integration, the gate substitutes the smoothed intra-window history at the *input* level, so the recurrent integration only ever sees frames it deems consistent with the local trend. This adds two scalar parameters beyond the plain-GRU.

(4) *Reliability-gated GRU*. The mixing $\tilde{e}_j$ and the GRU step are identical to (2); only the per-frame confidence α_j is produced by a learned scorer s_ψ rather than a fixed cosine statistic. With $d_j = |e_j - \bar{e}_{j-1}|$, $p_j = e_j \odot \bar{e}_{j-1}$, $c_j = \cos(e_j, \bar{e}_{j-1})$, and $\delta_j = \|d_j\|_2 / (\|\bar{e}_{j-1}\|_2 + \|e_j\|_2 + \varepsilon)$,

$$\alpha_j = \sigma(s_\psi([e_j; \bar{e}_{j-1}; d_j; p_j; c_j; \delta_j])), \quad (6)$$

where s_ψ is a 2-layer MLP (hidden 128, ReLU). The gate sees per-channel disagreement signals rather than a single scalar similarity, at the cost of $\sim 0.52\text{M}$ extra parameters.

(5) *Reliability-aware soft-fusion GRU*. The soft-fusion variant decouples detection from recurrent control. A GRU first reads the raw embedding sequence to produce a temporal memory m_t , while the same relation-based scorer estimates current-frame reliability. Instead of rewriting every frame

before the GRU, the reliability controls only the final fusion between current embedding and memory:

$$\begin{aligned} m_t &= \text{GRU}(e_{t-K+1}, \dots, e_t), \\ \beta_t &= \beta_{\min} + (1 - \beta_{\min}) \sigma(s_\psi(\cdot)), \\ \hat{e}_t &= \beta_t e_t + (1 - \beta_t) m_t, \end{aligned} \quad (7)$$

with $\beta_{\min}=0.5$. Thus the controller never fully discards the current observation, even when the reliability scorer shifts weight toward temporal memory.

C. Training

All learned variants warm-start from the public ObjectReact checkpoint and train end-to-end with the original waypoint-regression loss (MSE over $T=10$ waypoints, with cos/sin heading channels). We regenerate the data so each sample is a length- K window; the target for a window ending at t is the expert waypoint at t . For the full-frame zero diagnostic, each recurrent GRU architecture is trained once with noise_p=0.0 and once with noise_p=0.2, where one random frame embedding is zeroed in an augmented sample. All runs use gnm_temporal, $K=6$, Adam with learning rate 7×10^{-4} , batch size 128, 10 epochs, seed 0, the bigger_bot_0.3-sh_0.4 split, and a single RTX 4090. The later soft-fusion checkpoint is a conservative fine-tune from the reliability-gated GRU, using patch-mixed gate supervision and distillation to decouple reliability detection from the final control embedding.

IV. EXPERIMENTS AND EVALUATION

A. Setup

We evaluate on the HM3D [8] InstanceImageNav [3] validation set (hard difficulty) in Habitat-Sim, using ground-truth topometric goals, over the four ObjectReact tasks: *Imitate* (teach-and-repeat), *Alt-Goal* (reach a seen-but-unvisited goal), *Shortcut* (cut across a longer mapping trajectory), and *Reverse* (traverse the route backward). We report Success Rate (SR), SPL, and Soft-SPL (SSPL). Ground-truth perception isolates the controller from segmentation and matching errors.

All variants are evaluated under a single **full protocol** (evaluate_objecreact.py: step_idx=3, end_idx=108, max_steps=300), with blacklist filtering and Alt-Goal included, giving effective episode counts of 33, 23, 26, and 30 for Imitate, Alt-Goal, Shortcut, and Reverse. Goals are provided as ground-truth topometric nodes.

The experiments follow one narrative rather than one noise source. We first measure clean behavior so that robustness gains are not confused with lower nominal performance. We then use full-frame zero-out as a deliberately harsh diagnostic: it asks whether a temporal module can recover when one input frame is entirely invalid, and whether a learned gate actually marks that frame as unreliable. After that mechanism check, Sec. V moves to the main multi-corruption robustness claim, using the ObjectReact-style per-segment outlier-cost dropout and complementary localized patch failures.

B. Experiment 1: controlled clean evaluation

We first evaluate both matched training sets without inference-time corruption. Within each table, the three architectures share initialization, training data, seed, temporal window, and training corruption probability.

TABLE I
CONTROLLED CLEAN EVALUATION WITH noise_p=0.0 FOR ALL THREE LEARNED VARIANTS. PER-TASK VALUES ARE SR (%); THE FINAL COLUMN REPORTS AVERAGED SR/SPL/SSPL.

Method	Imi.	Alt.	Short.	Rev.	Avg SR/SPL/SSPL
Plain GRU	75.8	65.2	61.5	66.7	67.3 / 67.3 / 75.7
Gated GRU (cosine)	78.8	52.2	57.7	60.0	62.2 / 62.2 / 70.8
Reliability-gated GRU	45.5	56.5	7.7	36.7	36.6 / 36.6 / 50.4

Without corruption augmentation, plain GRU is strongest overall and the reliability-gated model collapses on Shortcut. This is a fair architectural comparison, but it also shows that the learned reliability mechanism is not effective without corrupted training examples.

Table II reports the independently trained noise_p=0.2 set. Its clean averages are closely matched: the maximum Avg SPL difference is only 0.78 points, making the later noisy comparison less confounded by clean-performance differences.

TABLE II
CONTROLLED CLEAN EVALUATION WITH MATCHED TRAINING CORRUPTION (noise_p=0.2 FOR ALL THREE LEARNED VARIANTS). PER-TASK VALUES ARE SR (%); THE FINAL COLUMN REPORTS AVERAGED SR/SPL/SSPL.

Method	Imi.	Alt.	Short.	Rev.	Avg SR/SPL/SSPL
Plain GRU	75.8	52.2	57.7	56.7	60.6 / 60.6 / 72.4
Gated GRU (cosine)	72.7	47.8	61.5	63.3	61.4 / 61.4 / 72.5
Reliability-gated GRU	60.6	65.2	50.0	66.7	60.6 / 60.6 / 72.1

C. Experiment 2: extreme full-frame inference corruption

At each control step, the current WayObject Costmap is independently replaced by a zero map with probability 0.2 *before* it enters temporal history. A fixed seed and episode identifier generate the same corruption schedule for every method. Logged corruption rates range from 19.41% to 20.20% across all 24 runs. This is an extreme diagnostic rather than the final perception-failure model: it creates fully invalid costmaps that are outside the ObjectReact per-segment augmentation distribution.

TABLE III
INFERENCE-NOISE EVALUATION FOR MODELS TRAINED WITH noise_p=0.0. PER-TASK VALUES ARE SR (%); THE FINAL COLUMN REPORTS AVERAGED SR/SPL/SSPL.

Method	Imi.	Alt.	Short.	Rev.	Avg SR/SPL/SSPL
Plain GRU	72.7	56.5	50.0	53.3	58.1 / 58.1 / 69.4
Gated GRU (cosine)	66.7	39.1	57.7	63.3	56.7 / 56.7 / 68.4
Reliability-gated GRU	30.3	47.8	0.0	20.0	24.5 / 24.5 / 39.9

Without training corruption, plain GRU has the best absolute noisy result, while cosine gating has the smallest clean-to-noisy degradation (-5.46 Avg SPL). Reliability gating is weak even on clean evaluation and degrades further under inference corruption (-12.05 Avg SPL).

TABLE IV
INFERENCE-NOISE EVALUATION FOR MODELS TRAINED WITH
noise_p=0.2. PER-TASK VALUES ARE SR (%); THE FINAL COLUMN
REPORTS AVERAGED SR/SPL/SSPL.

Method	Imi.	Alt.	Short.	Rev.	Avg SR/SPL/SSPL
Plain GRU	69.7	47.8	57.7	60.0	58.8 / 58.8 / 69.4
Gated GRU (cosine)	66.7	39.1	53.9	60.0	54.9 / 54.9 / 68.7
Reliability-gated GRU	63.6	60.9	46.2	70.0	60.2 / 60.2 / 74.0

Reliability gating becomes robust in this full-frame diagnostic after corruption augmentation. It reaches 60.2 Avg SPL under corruption, 1.36 points above the matched plain GRU and 5.25 above the matched cosine-gated GRU. It also retains its clean Avg SPL within 0.46 points. The gains concentrate on Alt-Goal and Reverse, whereas plain GRU remains strongest on Imitate and Shortcut. We do not treat this as the final realism claim; Sec. V revisits robustness with the paper-aligned segment-level corruption.

Architecture and augmentation interact strongly. Cosine gating loses 6.44 Avg SPL from clean to noisy evaluation, whereas the reliability-gated architecture loses only 0.46. This establishes a behavioral difference between the two architectures, but does not by itself establish that the reliability score is a calibrated failure detector. We test that mechanism directly in Sec. IV-E.

D. Complete 2×2 comparison

Table V condenses the complete controlled design into one fair comparison. Every cell uses the same evaluation protocol and differs only in training and inference corruption probability. The table exposes the key interaction: reliability gating is the weakest method when trained without corruption, but becomes the strongest under inference corruption after matched augmentation in the full-frame zero-out setting. Plain GRU changes little across training conditions, whereas cosine gating does not benefit from this corruption training.

E. Experiment 3: does the gate detect corrupted frames?

The behavioral results motivate a stricter mechanism test. For both reliability checkpoints (trained with corruption probability 0.0 and 0.2), we repeat all four noisy evaluations while logging the full $K=6$ gate vector and its current-frame value α_t at every control step. The inference-noise injector independently records whether the current costmap was zeroed, providing a direct binary failure label. The diagnostic runs exactly reproduce the original navigation outcomes: success status and final distance match for all 111 non-blacklisted episodes shared with the original reliability evaluation.

Because a lower α_t is intended to indicate lower reliability, we use $1 - \alpha_t$ as the corruption-detection score. We report

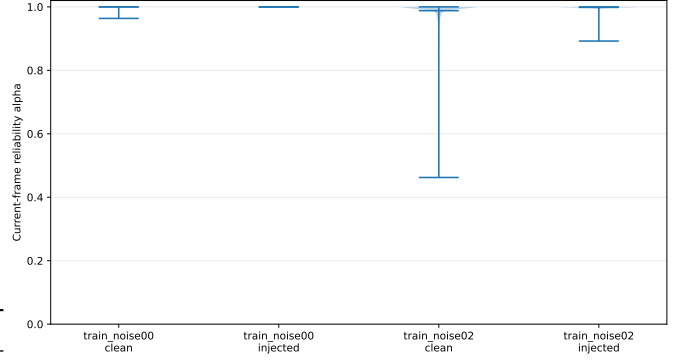


Fig. 2. Current-frame gate values for clean and injected steps. The noise-free checkpoint saturates near one. After corruption training, clean steps have greater variance and a *lower* mean alpha than injected steps—the opposite of the intended reliability semantics.

pooled and per-task ROC-AUC and average precision (AP), with 95% confidence intervals from 2,000 episode-cluster bootstrap samples (seed 42). Table VI shows the pooled result.

The learned score does not behave as a reliability detector. Both pooled ROC-AUC values are below 0.5, and every per-task ROC-AUC is between 0.209 and 0.333. The unaugmented gate is almost fully saturated at one. Corruption training increases score variation, but in the wrong direction: injected frames receive mean alpha 0.9984, compared with 0.9880 on clean frames (Fig. 2). Its AP of 0.136 is also below the injected-frame prevalence. Consequently, the navigation improvement in Table V cannot be attributed to the scorer explicitly identifying and suppressing zero-costmap frames. A more defensible interpretation is that task-loss training changes the combined gate-GRU dynamics and recurrent representation, while leaving the scalar gate uncalibrated. Explicit corruption supervision or a calibration loss is needed before α_t can be interpreted as reliability; the next experiment tests the simplest version and shows that calibrated detection is not sufficient for navigation.

F. Experiment 4: can explicit gate supervision fix the semantics?

The failed detector analysis raises a natural follow-up: perhaps the architecture is right, but the training signal is missing. To test this, we fine-tuned the corruption-trained reliability model for three epochs with explicit per-frame labels. Each non-initial frame is independently zeroed with probability 0.2, and a weighted binary cross-entropy term trains $1 - \alpha_t$ to predict the injected corruption label. The fine-tune starts from the original reliability checkpoint, uses learning rate 10^{-4} , gate-loss weight 0.5, positive weight 4.0, and updates the running history with the gated frame. We select the checkpoint with lowest validation total loss.

The detector objective succeeds completely: pooled ROC-AUC and AP both reach 1.000 on 26,842 logged evaluation steps. Mean alpha is 0.99951 for clean frames and 0.000079 for injected frames. Navigation, however, collapses (Table VII).

TABLE V

COMPLETE 2×2 TRAINING-NOISE BY INFERENCE-NOISE COMPARISON. VALUES ARE AVG SPL OVER IMITATE, ALT-GOAL, SHORTCUT, AND REVERSE. WITHIN EACH COLUMN, ALL ARCHITECTURES USE THE SAME TRAINING CORRUPTION, INFERENCE CORRUPTION, DATA, SEED, TEMPORAL WINDOW, AND EVALUATION PROTOCOL.

Method	Train noise 0.0		Train noise 0.2	
	Eval 0.0	Eval 0.2	Eval 0.0	Eval 0.2
Plain GRU	67.30	58.14	60.57	58.80
Gated GRU (cosine)	62.16	56.70	61.35	54.91
Reliability-gated GRU	36.56	24.52	60.62	60.16

TABLE VI

POOLED GATE-DETECTION RESULTS UNDER INFERENCE CORRUPTION PROBABILITY 0.2. POSITIVE LABELS ARE INJECTED ZERO-COSTMAP FRAMES AND THE DETECTION SCORE IS $1 - \alpha_t$. CONFIDENCE INTERVALS USE EPISODE-CLUSTER BOOTSTRAP. AP SHOULD BE COMPARED WITH THE APPROXIMATELY 19.5% POSITIVE RATE.

Train noise	ROC-AUC	AP	$\bar{\alpha}_{\text{clean}}$	$\bar{\alpha}_{\text{inj.}}$
0.0	0.255 [0.239, 0.271]	0.191 [0.185, 0.196]	1.0000	1.0000
0.2	0.312 [0.285, 0.337]	0.136 [0.130, 0.143]	0.9880	0.9984

TABLE VII

BLACKLIST-FILTERED NAVIGATION SPL AFTER THREE-EPOCH SUPERVISED GATE FINE-TUNING. THE SAME CHECKPOINT IS EVALUATED WITHOUT AND WITH INFERENCE CORRUPTION PROBABILITY 0.2.

Task	Eval noise 0.0	Eval noise 0.2
Imitate	42.42	48.48
Alt-Goal	47.83	52.17
Shortcut	26.92	19.23
Reverse	30.00	33.33
Average	36.79	38.31

The original corruption-trained reliability model reaches 60.62 clean and 60.16 noisy Avg SPL, so supervised fine-tuning loses 23.83 and 21.86 points respectively. Because performance is already poor when no inference corruption is present, the failure is not merely overreaction to injected frames. The result demonstrates a mechanism–control mismatch: a gate can become a perfect synthetic failure classifier while the coupled recurrent policy becomes a substantially worse navigator. For this architecture, calibrated detection is therefore not sufficient evidence of robust control. Future detector objectives must preserve the control representation, for example by freezing the navigation policy, decoupling detection from feature replacement, or regularizing the new policy against the original action predictions.

G. Experiment 5: matched qualitative rollouts

We next compare the corruption-trained Plain GRU and Reliability-Gated GRU under the same inference-noise probability 0.2. For each task, we automatically select a non-blacklisted episode where reliability gating succeeds and Plain GRU fails, prioritizing the largest reduction in final goal distance. This produces one strict matched pair per task; no episode is chosen from different start states or corruption

TABLE VIII

MATCHED QUALITATIVE CASES. PLAIN GRU REACHES THE 300-STEP LIMIT IN ALL FOUR EPISODES, WHILE RELIABILITY-GATED GRU SUCCEEDS. DISTANCES ARE METERS FROM THE GOAL AT TERMINATION.

Task	Plain dist.	Reliability dist.	Improvement
Imitate	2.729	0.993	1.735
Alt-Goal	6.875	0.978	5.897
Shortcut	8.710	0.991	7.719
Reverse	5.741	0.996	4.746

schedules.

The matched cases show that the difference is not a small threshold effect. On Alt-Goal, Shortcut, and Reverse, Plain GRU remains several meters from the goal after 300 steps, while the reliability model reaches within one meter. On Imitate, Plain GRU approaches the goal but fails to complete, whereas the reliability model succeeds at step 159. Supplementary side-by-side videos visualize the same four cases. These examples support the aggregate robustness result behaviorally, but they do not rescue the detector interpretation rejected by Sec. IV-E: the model can recover under corruption even though its scalar alpha is not calibrated. Sec. IV-F further shows that calibration alone is not enough to preserve the controller.

H. Diagnostic summary

The full-frame zero-out study is not the final perception-failure model; it is a controlled diagnostic that separates clean performance, recovery from a fully invalid frame, gate semantics, and the detection–control mismatch.

Clean behavior. Under matched noise_p=0.0 training, plain GRU is strongest and reliability gating performs poorly. Under matched noise_p=0.2 training, the three methods have nearly tied clean averages (60.6–61.4 SPL). Training corruption therefore has a large architecture-dependent effect and is essential for the learned reliability gate.

Full-frame robustness. Training corruption interacts strongly with architecture. For reliability gating, it raises noisy Avg SPL from 24.5 to 60.2 and changes the clean-to-noisy loss from 12.05 to 0.46 points. Plain GRU gains only 0.66 noisy SPL from training corruption, while cosine gating loses 1.79. The benefit is therefore specific to the jointly trained reliability-gated recurrent architecture, not a generic effect of augmentation. However, the gate diagnostic shows that this benefit is not implemented as the intended low-alpha corruption detector.

Detection versus control. Explicit gate supervision resolves the detector problem but breaks navigation: ROC-AUC/AP reach 1.000/1.000, while clean/noisy Avg SPL fall to 36.8/38.3. Thus the report should not argue that the reliability gate is useful because it has discovered a human-interpretable failure probability. The safer claim is that task-trained temporal recurrence can learn a robust control representation, and that a separate design is needed if calibrated reliability is also required. This distinction is important for the next section: the full-frame diagnostic shows that temporal memory can rescue behavior, but it does not decide which aggregation mechanism is best for ObjectReact’s own object-level failure mode.

V. RESULTS AND DISCUSSION

Having used full-frame zero-out to analyze recovery and gate semantics, we now ask whether the same conclusions hold under costmap failures shaped like the ObjectReact pipeline itself. This section is therefore the main robustness result. It compares two complementary corruption geometries: independently resampled object-level outliers, which match the original ObjectReact augmentation, and localized patch failures, which model spatially coherent costmap errors.

a) *From invalid frames to object-level outliers.*: The earlier full-frame diagnostic zeroes a whole current costmap with probability 0.2. At deployment this corresponds to a per-step independent draw from the two-component distribution $\{K \text{ clean}, K-1 \text{ clean}+1 \text{ all-zero}\}$. On revisiting the original ObjectReact training pipeline we found that the costmap-level augmentation *actually present at training time* is PER-TURB_MASK_PLS (§3.3 of the original paper [2]), which independently replaces a fraction $\rho_{\text{train}}=0.3$ of the *segments* in every frame by the fixed outlier path-length value. In other words, every training frame is partially corrupted, and no training frame is ever fully zero. The zero-out diagnostic therefore probes a useful but intentionally out-of-distribution regime, whereas the following experiment tests the corruption family the controller was trained to tolerate.

To remove this confound, the main object-level evaluation uses the *paper-aligned* corruption mechanism at inference: at every controller step a fraction ρ of the segments in the costmap is independently replaced by the outlier cost. At $\rho=\rho_{\text{train}}=0.3$ this matches the training distribution exactly; $\rho \in \{0.5, 0.7\}$ constitutes a strict superset of the training distribution and probes out-of-distribution robustness. The aggregator checkpoints are unchanged from the clean-perception experiments and no inference-time finetuning is performed.

Evaluation uses the four ObjectReact tasks (Imitate, Alt-Goal, Shortcut, Reverse) on the 36 HM3D-val annotated scenes with the standard episode blacklist applied; filtered episode counts $n_{\text{Imi.}}=33$, $n_{\text{Alt.}}=23$, $n_{\text{Sho.}}=26$, $n_{\text{Rev.}}=30$. Success Rate, SPL and Soft-SPL follow the definitions of §IV-B.

A. Object-level outliers: main result at $\rho=0.5$

Table IX reports the three metrics at the representative stress point $\rho=0.5$ ($1.67\times$ the training ratio). Five methods

are compared: the four temporal aggregators from §III (EMA, Plain GRU, Cosine-Gated GRU, Reliability-Gated GRU) and the *per-step mean-pool baseline*, which runs the unmodified ObjectReact controller on each frame without temporal aggregation.

The Cosine-Gated GRU strictly beats the baseline on every task and every metric, more than doubling the baseline Success Rate on Imitate ($21.21 \rightarrow 42.42$, $+21.21$ SR) and Reverse ($10.00 \rightarrow 26.67$, $+16.67$ SR), and recovering Shortcut from the baseline collapse to 0% to a non-trivial 19.23%.

B. Robustness across the noise rate

Table X extends the evaluation to three rates spanning the training point ($\rho=0.3$, in-distribution), a moderate out-of-distribution level ($\rho=0.5$) and a heavy out-of-distribution level ($\rho=0.7$). The Cosine-Gated GRU is the strongest aggregator at every (task, ρ) cell — 12/12 Success-Rate wins against the baseline — and degrades gracefully under increasing ρ .

In contrast the per-step baseline and the EMA-only aggregator drop sharply on Reverse ($16.67 \rightarrow 3.33$ SR for the baseline; $20.00 \rightarrow 3.33$ for EMA from $\rho=0.3$ to $\rho=0.7$), and both fail outright (SR=0) on Shortcut at every $\rho \leq 0.5$. Plain GRU and Reliability-Gated GRU sit between EMA and the Cosine-Gated variant: their recurrent memory captures part of the redundancy across frames, but without the cosine-similarity gate they amplify stale information when the recent window is dominated by outliers.

C. Analysis

Per-segment dropout is a sharper test of temporal aggregation than the full-frame zero-out diagnostic because no individual frame in the $K=6$ window is fully invalid: every frame retains $1-\rho$ of its segments, so the aggregator must integrate partially reliable evidence rather than simply ignore an all-zero frame. The single-frame mean-pool baseline cannot exploit this redundancy — each step it sees a fresh costmap where ρ of segments carry no useful path-length information, and on the tightly-scripted Shortcut task this is enough to drive its Success Rate to zero.

The aggregators see the same per-frame noise but obtain $K=6$ independent draws of which segments are dropped (the noise is re-sampled every step), so for any fixed segment position the probability that all K frames in the window are simultaneously corrupted is only $\rho^K = 0.5^6 \approx 1.6\%$ at $\rho=0.5$ and $\rho^K = 0.3^6 \approx 0.07\%$ at $\rho=0.3$, a substantial reduction of the effective per-segment outlier rate. The Cosine-Gated GRU realises this gain in two complementary ways. First, its cosine-similarity gate down-weights frames whose embedding has drifted far from the running EMA history (the gate output $\alpha_j \rightarrow 0$ on inconsistent frames), so a window with several heavily-corrupted frames does not pull the recurrent state off-distribution. Second, the gated recurrent update preserves the most informative past embedding when the current step is dominated by outliers, exactly the behaviour Reverse and Shortcut benefit from. Plain GRU achieves the K -frame averaging effect but lacks the rejection step; EMA

TABLE IX
ROBUSTNESS UNDER PER-SEGMENT OUTLIER-COST DROPOUT AT THE REPRESENTATIVE STRESS RATIO $\rho=0.5$ (TRAINING DISTRIBUTION: $\rho=0.3$). HM3D-VAL, BLACKLIST-FILTERED ($n_{\text{Imi.}}=33$, $n_{\text{Alt.}}=23$, $n_{\text{Sho.}}=26$, $n_{\text{Rev.}}=30$). PER-TASK BEST NON-BASELINE SUCCESS RATE IN BOLD.

Method	Success Rate (%)				SPL (%)				Soft-SPL (%)			
	Imi.	Alt.	Sho.	Rev.	Imi.	Alt.	Sho.	Rev.	Imi.	Alt.	Sho.	Rev.
Baseline (mean-pool)	21.21	13.04	0.00	10.00	21.21	13.04	0.00	9.90	35.00	17.14	6.90	26.89
EMA	18.18	8.70	0.00	16.67	18.18	8.70	0.00	16.57	34.72	21.24	7.36	25.50
Plain GRU	33.33	17.39	15.38	10.00	33.33	17.38	15.38	10.00	42.10	26.29	30.35	22.56
Cosine-Gated GRU	42.42	21.74	19.23	26.67	42.40	21.73	19.23	26.60	48.61	27.20	28.96	43.78
Reliability-Gated GRU	36.36	13.04	15.38	20.00	36.36	13.04	15.38	19.96	42.48	18.98	26.39	32.37

TABLE X
SUCCESS RATE (%) AS A FUNCTION OF THE PER-SEGMENT OUTLIER RATIO ρ . TRAINING DISTRIBUTION IS $\rho=0.3$; $\rho \in \{0.5, 0.7\}$ ARE STRICT OUT-OF-DISTRIBUTION SUPERSETS. PER-TASK BEST NON-BASELINE CELL IN BOLD.

Method	Imitate			Alt-Goal			Shortcut			Reverse		
	0.3	0.5	0.7	0.3	0.5	0.7	0.3	0.5	0.7	0.3	0.5	0.7
Baseline (mean-pool)	18.18	21.21	15.15	8.70	13.04	8.70	0.00	0.00	3.85	16.67	10.00	3.33
EMA	24.24	18.18	12.12	8.70	8.70	4.35	0.00	0.00	0.00	20.00	16.67	3.33
Plain GRU	45.45	33.33	24.24	17.39	17.39	13.04	11.54	15.38	0.00	16.67	10.00	6.67
Cosine-Gated GRU	39.39	42.42	48.48	17.39	21.74	13.04	19.23	19.23	7.69	30.00	26.67	26.67
Reliability-Gated GRU	48.48	36.36	39.39	17.39	13.04	8.70	15.38	15.38	3.85	20.00	20.00	16.67

averages but has no gating, which is why it tracks the baseline closely.

a) Take-away. Under the paper-aligned per-segment outlier-dropout corruption the Cosine-Gated GRU aggregator achieves a 12/12 task-rate sweep over the per-step baseline by Success Rate, with parallel gains on SPL and Soft-SPL. Performance degrades gracefully from the training ratio $\rho=0.3$ (in-distribution) to the heavy out-of-distribution ratio $\rho=0.7$. Under paper-aligned object-level perception failure, temporal aggregation helps, and the cosine-similarity gate is the mechanism that makes the recurrent variant robust across noise rates. The aggregator checkpoints in this section are exactly those produced by the clean-perception training stage, with the standard `pl_perturb_ratio=0.3` augmentation; no inference-mode finetuning is performed.

D. Spatially coherent failures: localized-patch stress test

Per-segment dropout is object-level and independently resampled. Real perception errors can also be spatially coherent: a region of the costmap may be missing, or it may persist from a previous frame. To test that different geometry, we ran a complementary localized-corruption check with two modes: `patch_dropout`, which zeros a random rectangular region of the current costmap, and `patch_stale`, which replaces that region with the previous frame’s values. Both use probability 0.2, patch area ratio 0.25, seed 42, and the same blacklist-filtered protocol as the main evaluation. Table XI reports averages over the four tasks.

This patch stress test does not overturn the per-segment conclusion; it qualifies it. Cosine gating is strongest for independently resampled per-segment outliers, whereas spatially coherent local failures favor parameter-free EMA and reliability-

aware soft fusion. The consistent message is that temporal redundancy helps ObjectReact, but the best aggregator depends on the structure of the perception failure. In particular, the original reliability-gated GRU should not be presented as a universally best or interpretable reliability detector.

VI. LIMITATIONS

We state the scope of these claims explicitly.

- **Synthetic corruption.** Full-frame zero-out, per-segment outlier dropout, and localized patch corruption are controlled and auditable, but none fully reproduces the structure of FastSAM/LightGlue errors.
- **Modest episode budget.** Each task has at most 36 raw episodes and fewer after blacklist filtering. Method margins should be treated as trends.
- **Single seed.** Learned variants use training seed 0; seed variance has not been measured.
- **Dataset failures.** Shortcut contains two known missing-data episodes and one non-blacklisted episode without a success status. All methods are evaluated with the same handling.
- **Detection-control trade-off.** The reliability-gated architecture is robust behaviorally in the full-frame diagnostic, but its scalar alpha is saturated and anti-correlated with injected failures under the intended interpretation. Supervised fine-tuning fixes the synthetic detector to ROC-AUC/AP 1.0 but reduces clean/noisy Avg SPL to 36.8/38.3, so the current architecture cannot provide both calibrated detection and strong control.
- **Qualitative selection.** The matched videos deliberately select episodes where the methods differ and therefore

TABLE XI
LOCALIZED PATCH CORRUPTION WITH BLACKLIST-FILTERED AVERAGES OVER IMITATE, ALT-GOAL, SHORTCUT, AND REVERSE. EACH ENTRY IS AVG SR / AVG SPL / AVG SOFT-SPL.

Method	Patch dropout	Patch stale
ObjectReact single frame	61.81 / 61.80 / 73.89	68.89 / 68.88 / 77.81
EMA	66.33 / 66.33 / 75.20	69.97 / 69.96 / 78.51
Plain GRU	58.68 / 58.67 / 69.89	59.76 / 59.76 / 69.02
Cosine-Gated GRU	55.44 / 55.44 / 69.85	62.44 / 62.44 / 73.23
Reliability-Gated GRU	63.61 / 63.60 / 71.54	61.43 / 61.43 / 72.00
Reliability-Aware Soft Fusion	67.70 / 67.70 / 77.88	69.75 / 69.75 / 79.29

illustrate failure modes rather than estimate their population frequency; aggregate rates are reported separately.

VII. REMAINING WORK

- 1) **Inferred-perception evaluation.** Replace ground-truth segmentation with FastSAM [12] and LightGlue [4] to test naturally structured perception failures.
- 2) **Multiple seeds and corruption levels.** Repeat training and evaluate sweeps over segment ratios, patch probabilities, and inference seeds to quantify variance and robustness curves.
- 3) **Detector-controller decoupling.** Preserve the original navigation policy while learning reliability, for example with a frozen controller, a separate detector head, soft interventions, or action-distillation regularization. Evaluate detection and navigation jointly rather than treating ROC-AUC as a proxy for control quality.

VIII. CONCLUSION

We targeted ObjectReact’s single-frame brittleness with lightweight temporal aggregation. Across the three corruption geometries, the same high-level pattern emerges with different winners. Full-frame zero-out is useful as a diagnostic: corruption-trained reliability-gated recurrence can recover from extreme invalid frames, but its scalar alpha does not identify injected failures and is not an interpretable reliability probability. Explicit supervision confirms that this is not a superficial training bug: a perfect synthetic detector can still produce a weak navigator. Under per-segment outlier-cost corruption, which matches the ObjectReact training-time augmentation, the Cosine-Gated GRU is the strongest recurrent aggregator across noise rates and wins all 12 task-rate comparisons against the per-step baseline. Under localized patch failures, EMA and reliability-aware soft fusion are strongest. The most defensible conclusion is therefore not that a learned reliability gate has solved perception failure, but that short-horizon temporal aggregation is a robust ObjectReact extension, detection and control must be evaluated separately, and the aggregation mechanism should be matched to the structure of the corruption.

ACKNOWLEDGMENTS

We build on the public ObjectReact codebase and the Habitat/HM3D ecosystem. Our repository is at <https://github.com/6chHenry/object-rel-nav>.

REFERENCES

- [1] Sourav Garg, Krishan Rana, Mehdi Hosseinzadeh, Lachlan Mares, Niko Suenderhauf, Feras Dayoub, and Ian Reid. Robohop: Segment-based topological map representation for open-world visual navigation. In *IEEE International Conference on Robotics and Automation*, 2024.
- [2] Sourav Garg, Dustin Craggs, Vineeth Bhat, Lachlan Mares, Stefan Podgorski, Madhava Krishna, Feras Dayoub, and Ian Reid. Objectreact: Learning object-relative control for visual navigation. In *Conference on Robot Learning*, 2025.
- [3] Jacob Krantz, Stefan Lee, Jitendra Malik, Dhruv Batra, and Devendra Singh Chaplot. Instance-specific image goal navigation: Training embodied agents to find object instances. *arXiv preprint arXiv:2211.15876*, 2022.
- [4] Philipp Lindenberger, Paul-Edouard Sarlin, and Marc Pollefeys. Light-Glue: Local feature matching at light speed. In *IEEE/CVF International Conference on Computer Vision*, pages 17627–17638, 2023.
- [5] Ajay Mandlekar, Danfei Xu, Josiah Wong, Soroush Nasiriany, Chen Wang, Rohun Kulkarni, Li Fei-Fei, Silvio Savarese, Yuke Zhu, and Roberto Martin-Martin. What matters in learning from offline human demonstrations for robot manipulation. In *Conference on Robot Learning*, 2021.
- [6] Piotr Mirowski, Razvan Pascanu, Fabio Viola, Hubert Soyer, Andrew J. Ballard, Andrea Banino, Misha Denil, Ross Goroshin, Laurent Sifre, Koray Kavukcuoglu, Dharshan Kumaran, and Raia Hadsell. Learning to navigate in complex environments. In *International Conference on Learning Representations*, 2017.
- [7] Stefan Podgorski, Sourav Garg, Mehdi Hosseinzadeh, Lachlan Mares, Feras Dayoub, and Ian Reid. TANGO: Traversability-aware navigation with local metric control for topological goals. *arXiv preprint arXiv:2509.08699*, 2025.
- [8] Santhosh K. Ramakrishnan, Aaron Gokaslan, Erik Wijmans, Oleksandr Maksymets, Alex Clegg, John Turner, Eric Undersander, Wojciech Galuba, Andrew Westbury, Angel X. Chang, Manolis Savva, Yili Zhao, and Dhruv Batra. Habitat-matterport 3d dataset (HM3D): 1000 large-scale 3d environments for embodied ai. *Advances in Neural Information Processing Systems Datasets and Benchmarks Track*, 2021.
- [9] Stephane Ross, Geoffrey Gordon, and Drew Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. In *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics*, pages 627–635, 2011.
- [10] Dhruv Shah, Ajay Sridhar, Arjun Bhorkar, Noriaki Hirose, and Sergey Levine. GNM: A general navigation model to drive any robot. In *IEEE International Conference on Robotics and Automation*, pages 7226–7233, 2023.
- [11] Dhruv Shah, Ajay Sridhar, Nitish Dashora, Kyle Stachowicz, Kevin Black, Noriaki Hirose, and Sergey Levine. ViNT: A foundation model for visual navigation. In *Conference on Robot Learning*, 2023.
- [12] Xu Zhao, Wenchao Ding, Yongqi An, Yinglong Du, Tao Yu, Min Li, Ming Tang, and Jinqiao Wang. Fast segment anything. *arXiv preprint arXiv:2306.12156*, 2023.